

7a. Multi – level linked list

–Delete At Position

–Program Analysis

Program: –

```
struct Node
{
    int data;
    struct Node *next; // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
};

Node *head = nullptr; // Global Head Pointer
// Helper: Create a new node with malloc
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}

void deleteAtPosition()
{
    int pos;
    cout << "Enter position to delete: ";
    cin >> pos;

    if (head == nullptr)
        return;
```

```

Node *temp = head;

if (pos == 1)
{
    head = head->next;
    free(temp);
    return;
}

for (int i = 1; i < pos - 1 && temp != nullptr; i++)
{
    temp = temp->next;
}

if (temp == nullptr || temp->next == nullptr)
{
    cout << "Position out of range." << endl;
    return;
}

Node *toDelete = temp->next;
temp->next = toDelete->next;
free(toDelete);
cout << "Deleted at position " << pos << endl;
}
...
int main()
{
    int choice;
    while (true)
    {
        cout << "7. Delete at Position (Main)" << endl;
        cin >> choice;

        switch (choice)
        {
            case 7:

```

```

        deleteAtPosition();
        break;
    default:
        cout << "Invalid choice" << endl;
    }
}
return 0;
}

```

Program Analysis

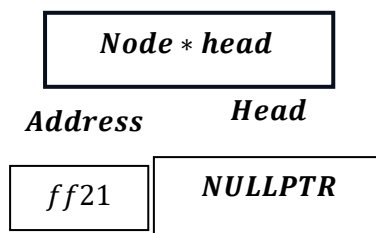
1)When List is Empty

If List is Empty, then:

```

if (head == nullptr)
    return;

```



Return void and exit from function.

2)When List is not Empty

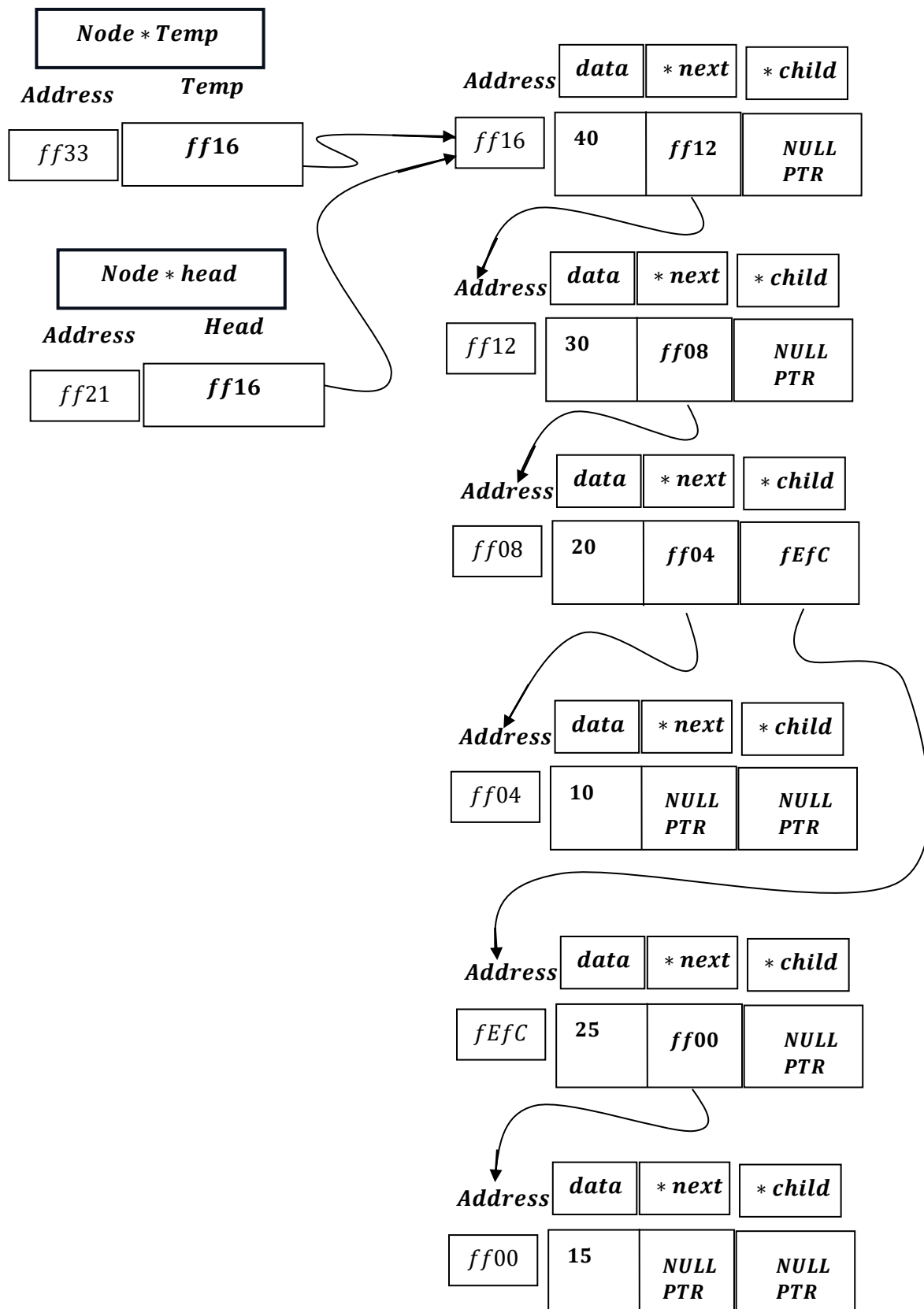
```
int pos;  
cout << "Enter position to delete: ";  
cin >> pos;
```

2.A.When Position is 1

Say, User Input ,Pos: 1

```
Node *temp = head;
```

Node * temp = head = ff16 .



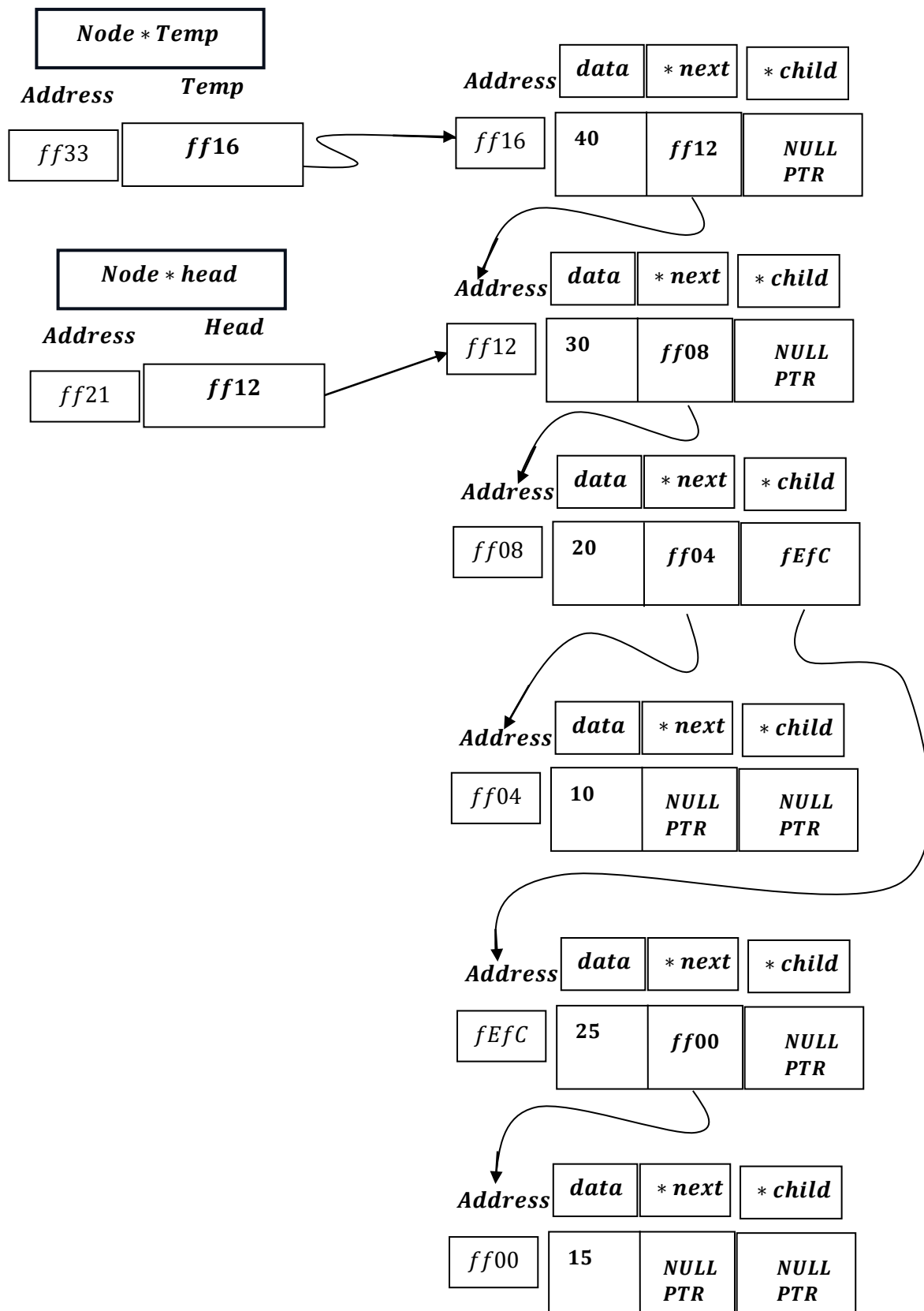
```
if (pos == 1)
{
    head = head->next;
    free(temp);
    return;
}
```

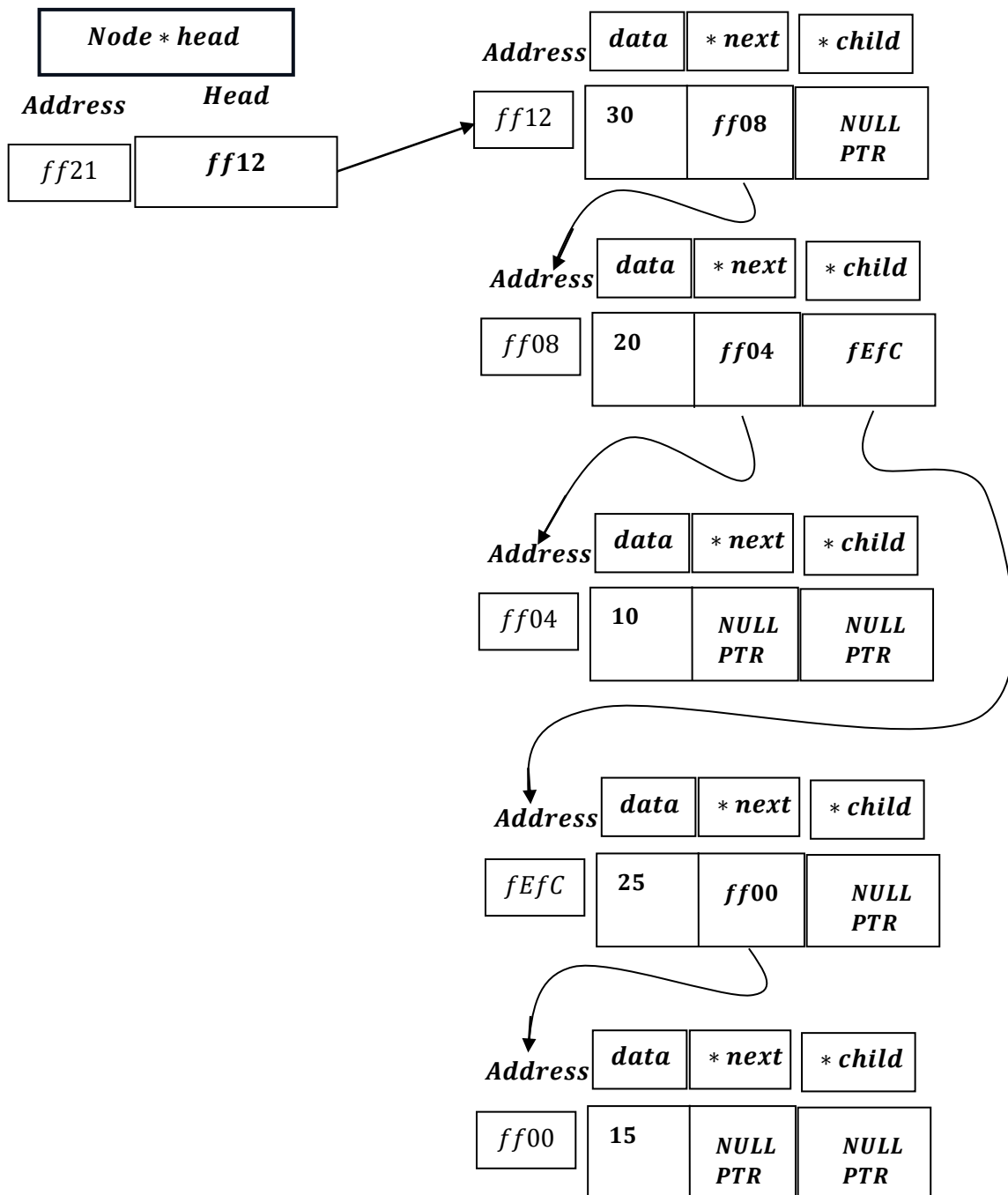
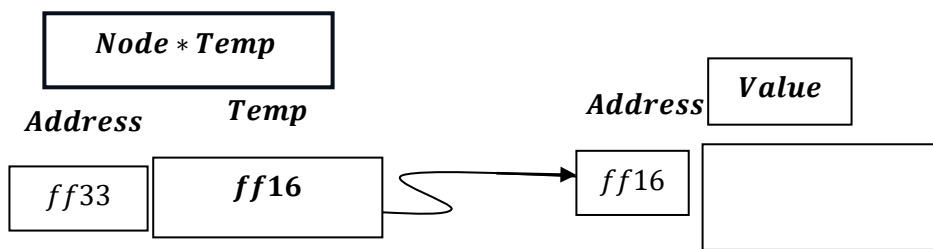
if pos = 1:

head = head → next = ff12

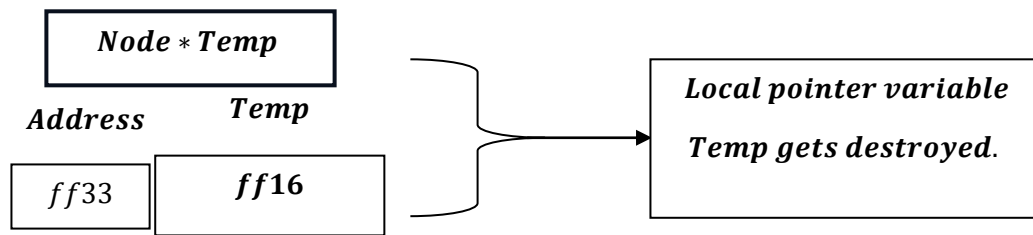
free(temp: ff16)

Return void and exit from function.

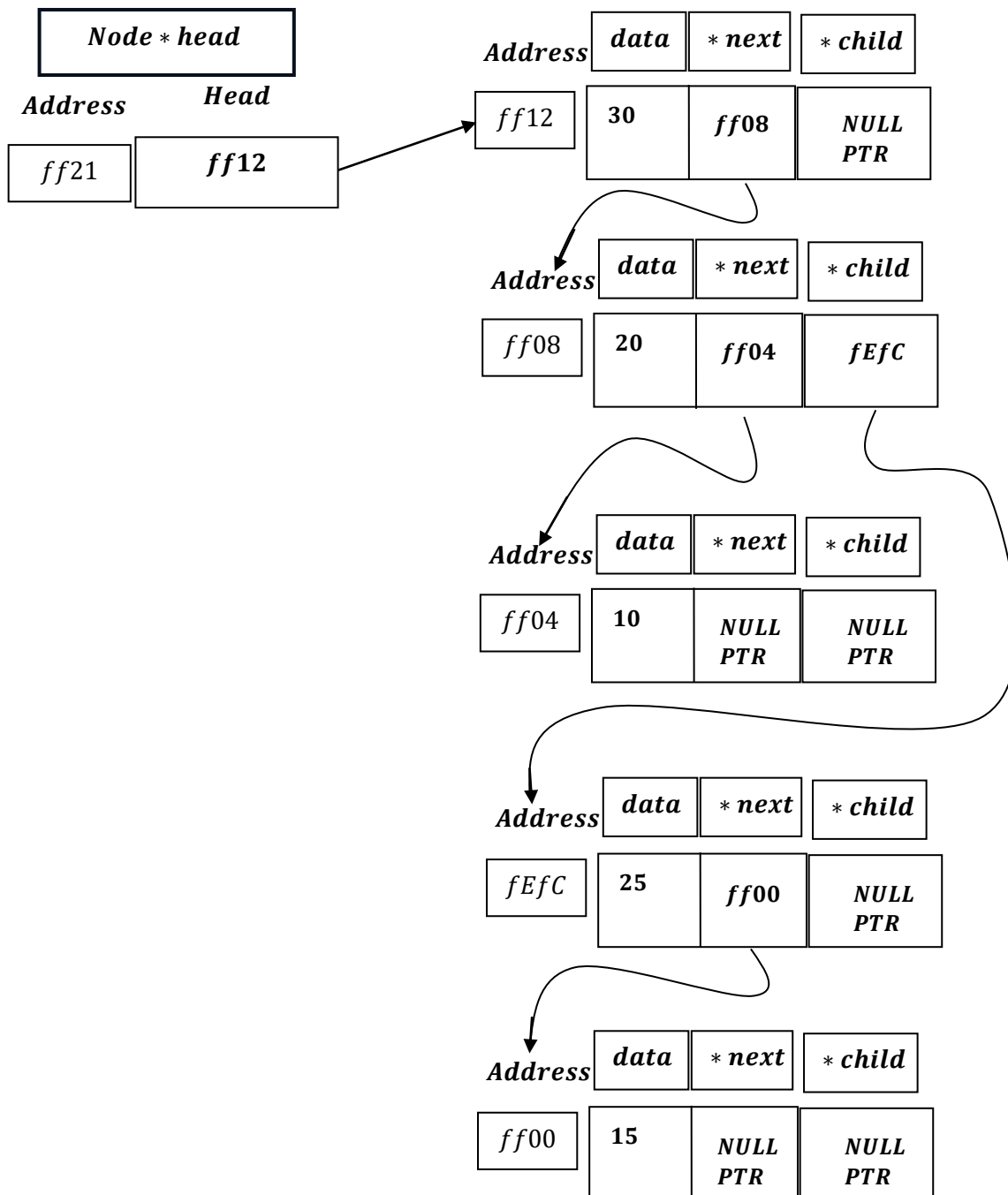




As, return void and function ends:



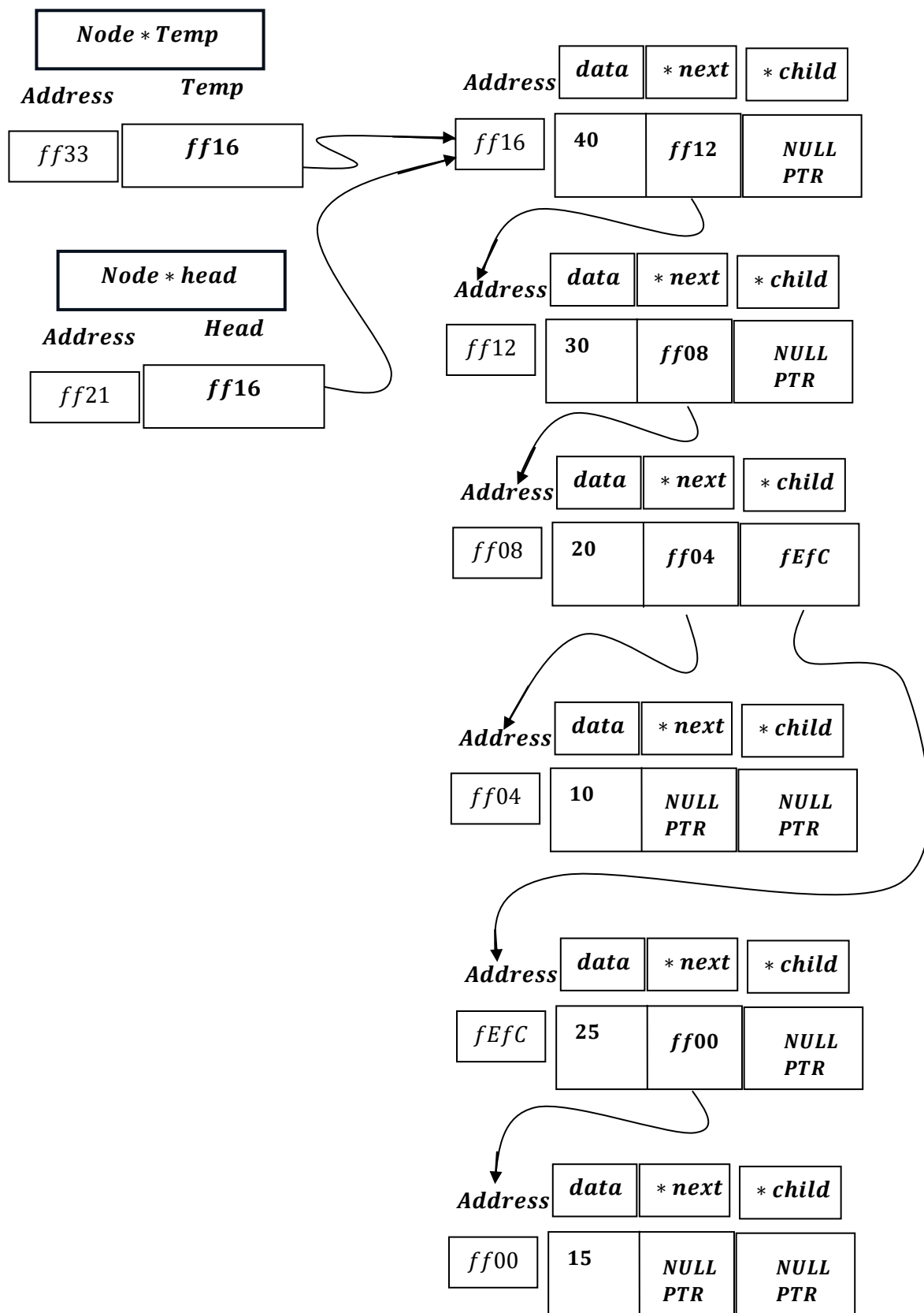
And, we are left with:



2.B.When Position is > 1

```
for (int i = 1; i < pos - 1 && temp != nullptr; i++)  
{  
    temp = temp->next;  
}
```

Say, pos = 3.

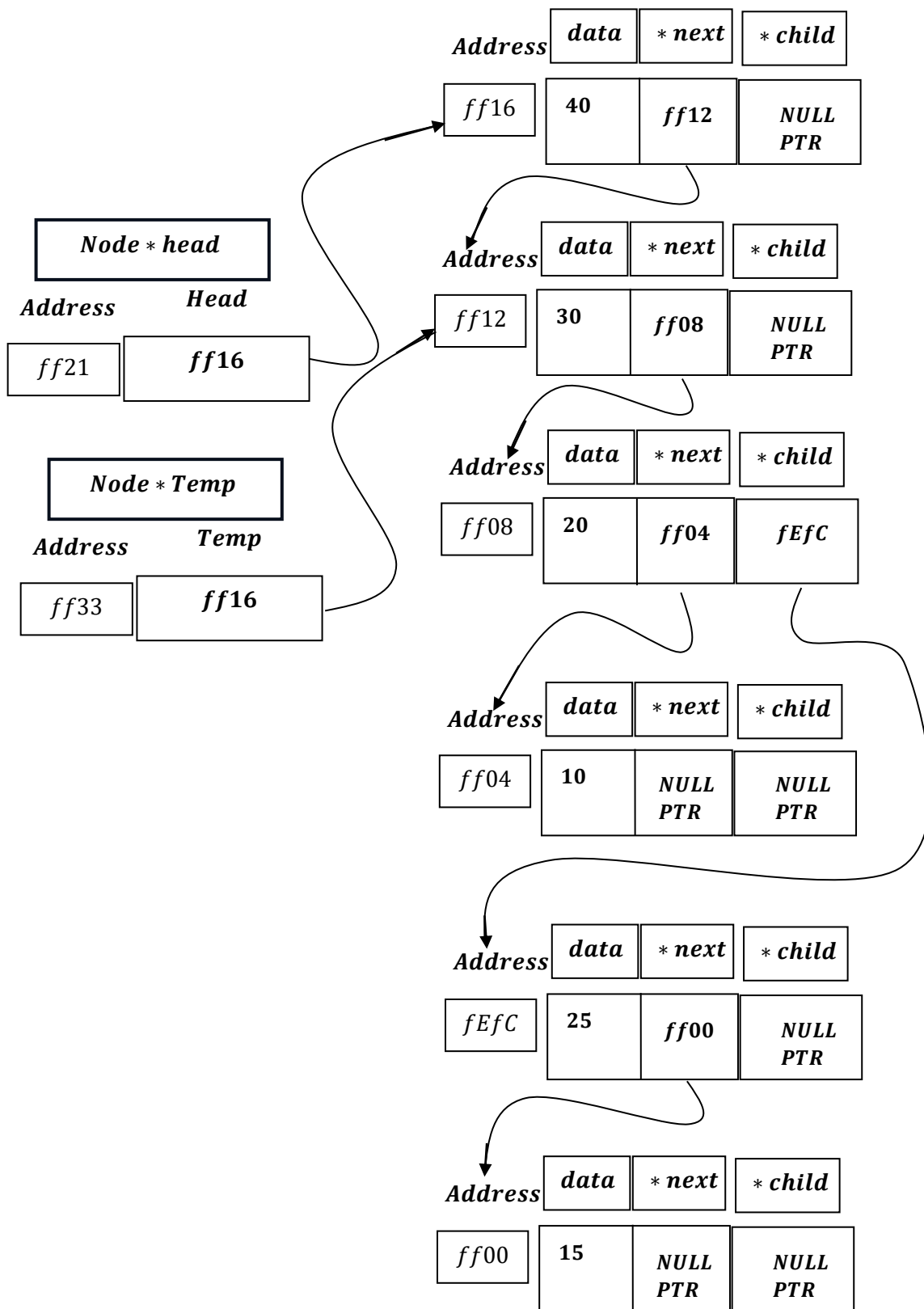


Now traverse until the Temp pointer points to the previous node of the node ,that is to get deleted:

Loop traversal:

int i = 1; i < 3 - 1[i < 2 or i = 1] AND TEMP ≠ NULL:

TEMP = TEMP → NEXT = ff12



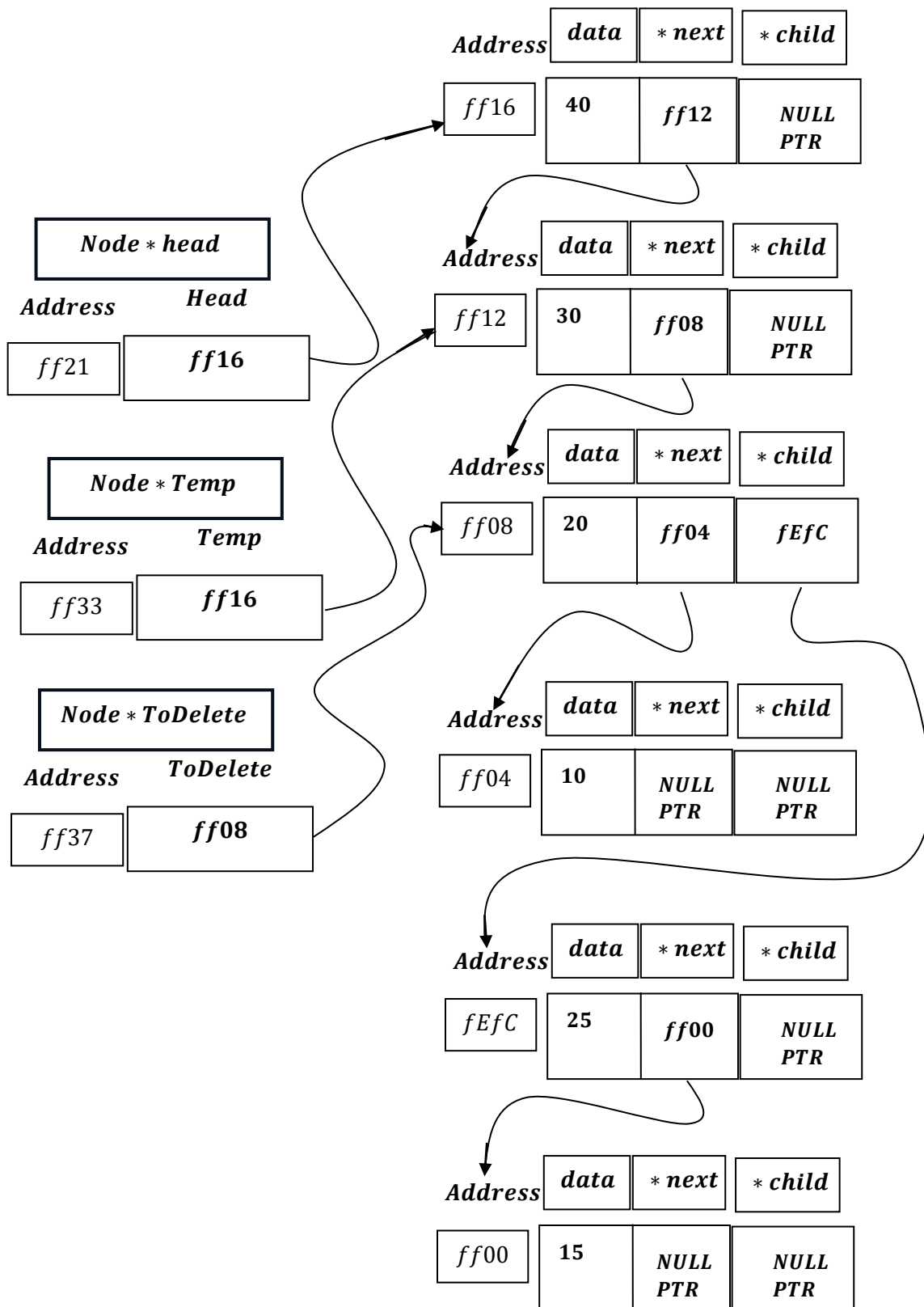
Update: $i = i + 1 = 2$

int i = 2; i < 3 - 1 [i < 2 or i = 1] AND TEMP ≠ NULL:

Condition is false , as Lower Bound > Upper Bound, loop exits.

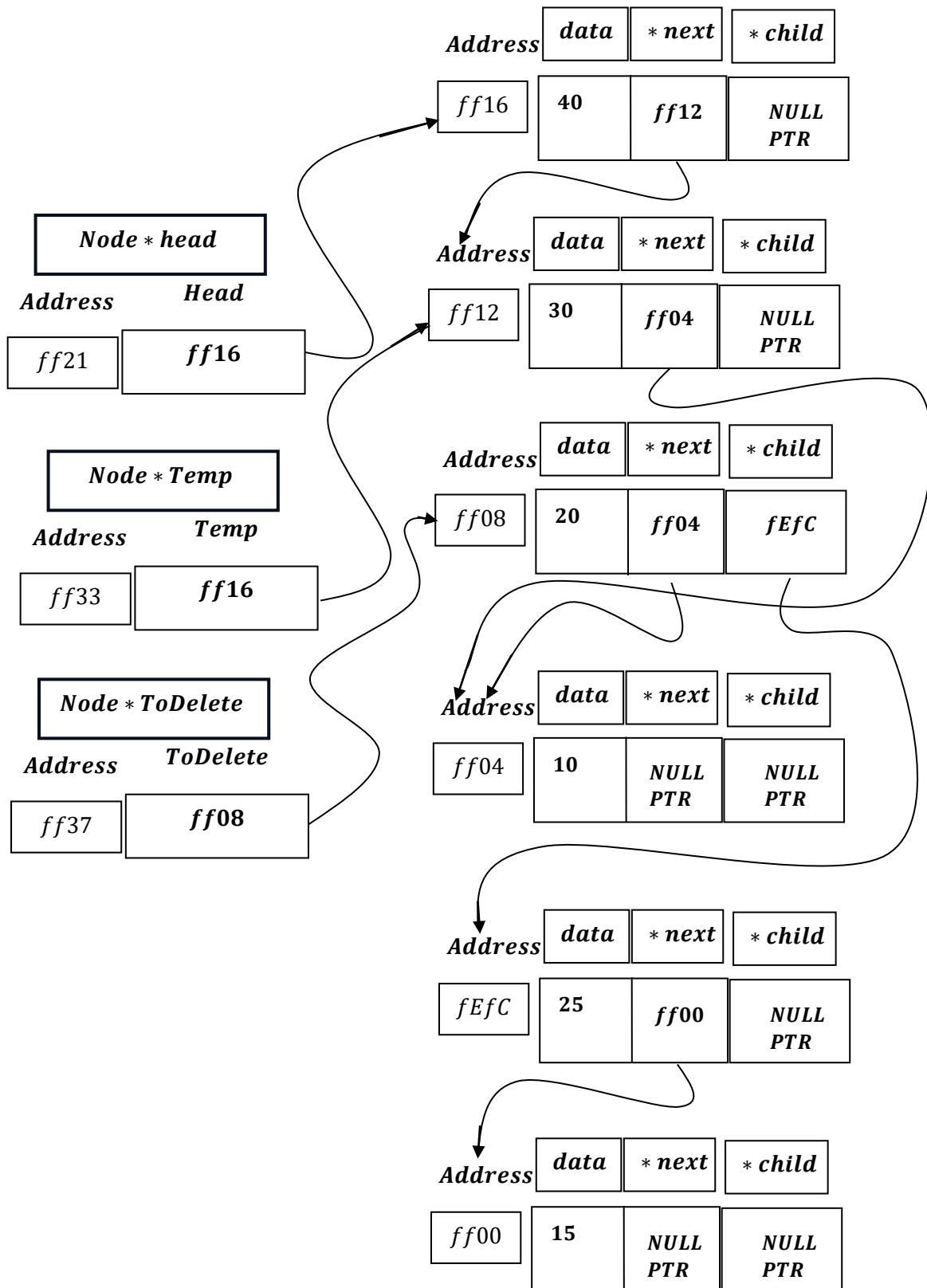
```
Node *toDelete = temp->next;
```

ToDelete = Temp → Next = ff08.

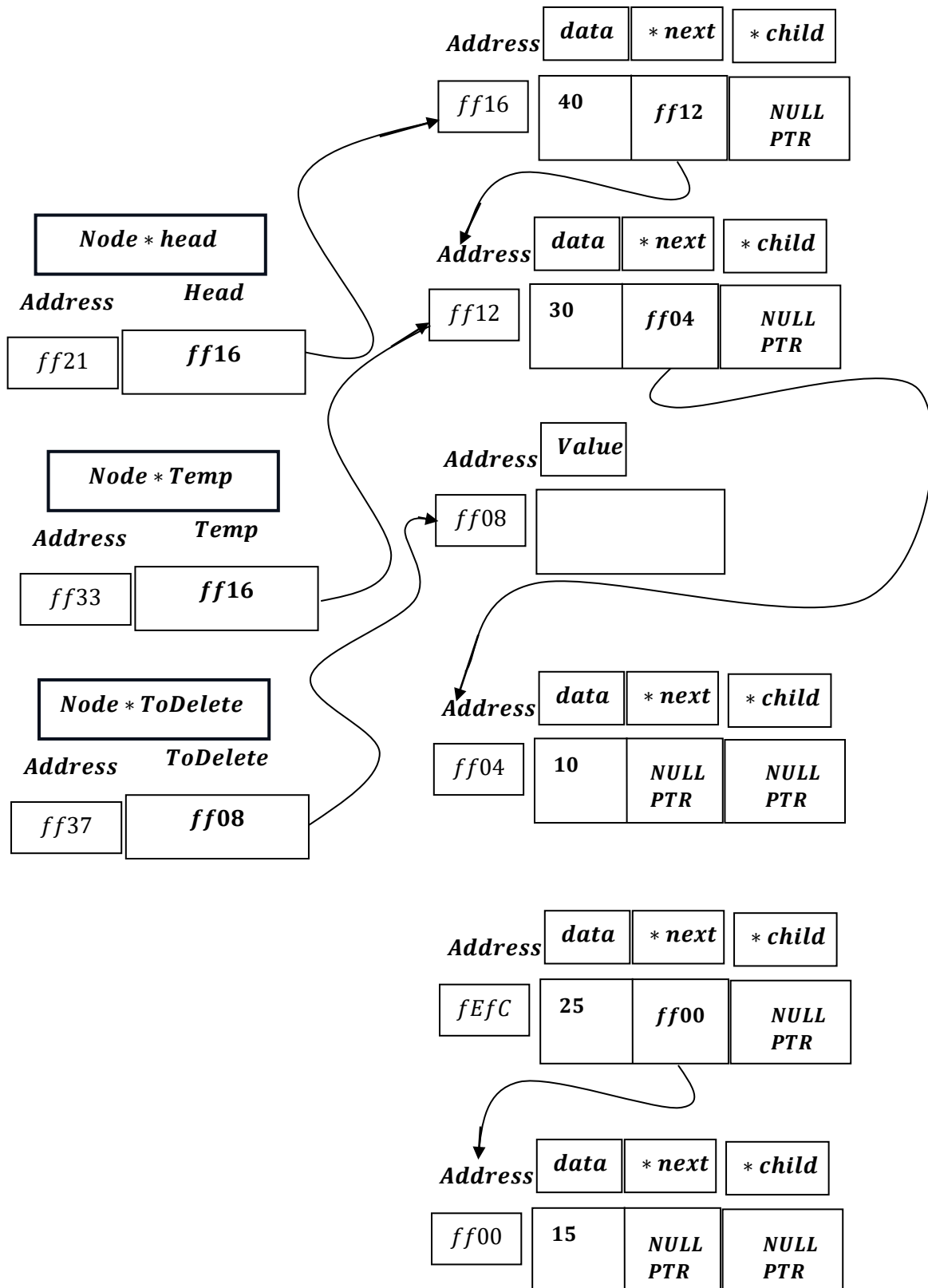


```
temp->next = toDelete->next;
```

TEMP → NEXT = ToDelete → Next = ff04



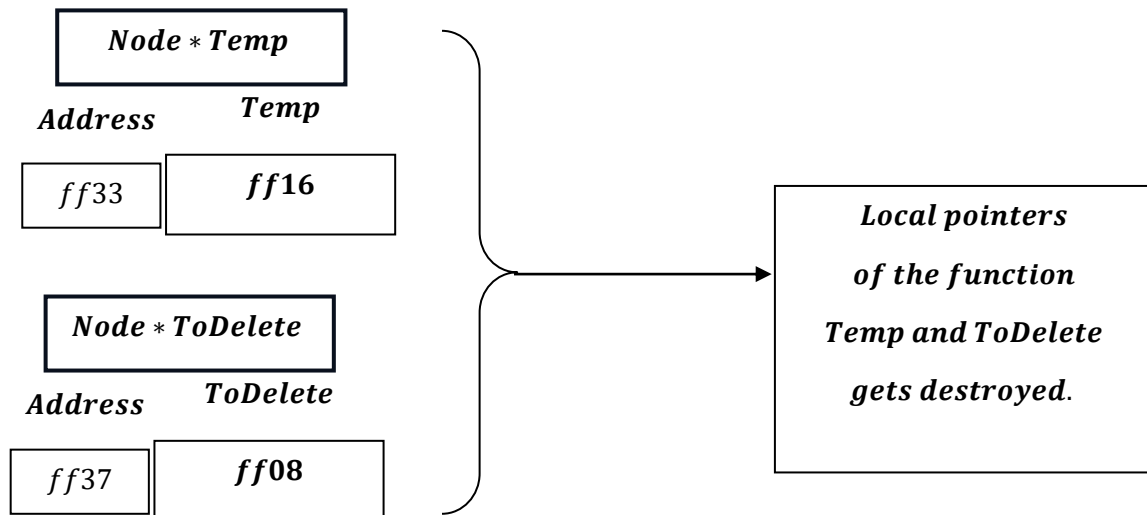

```
free(toDelete);  
free(toDelete: ff08)
```



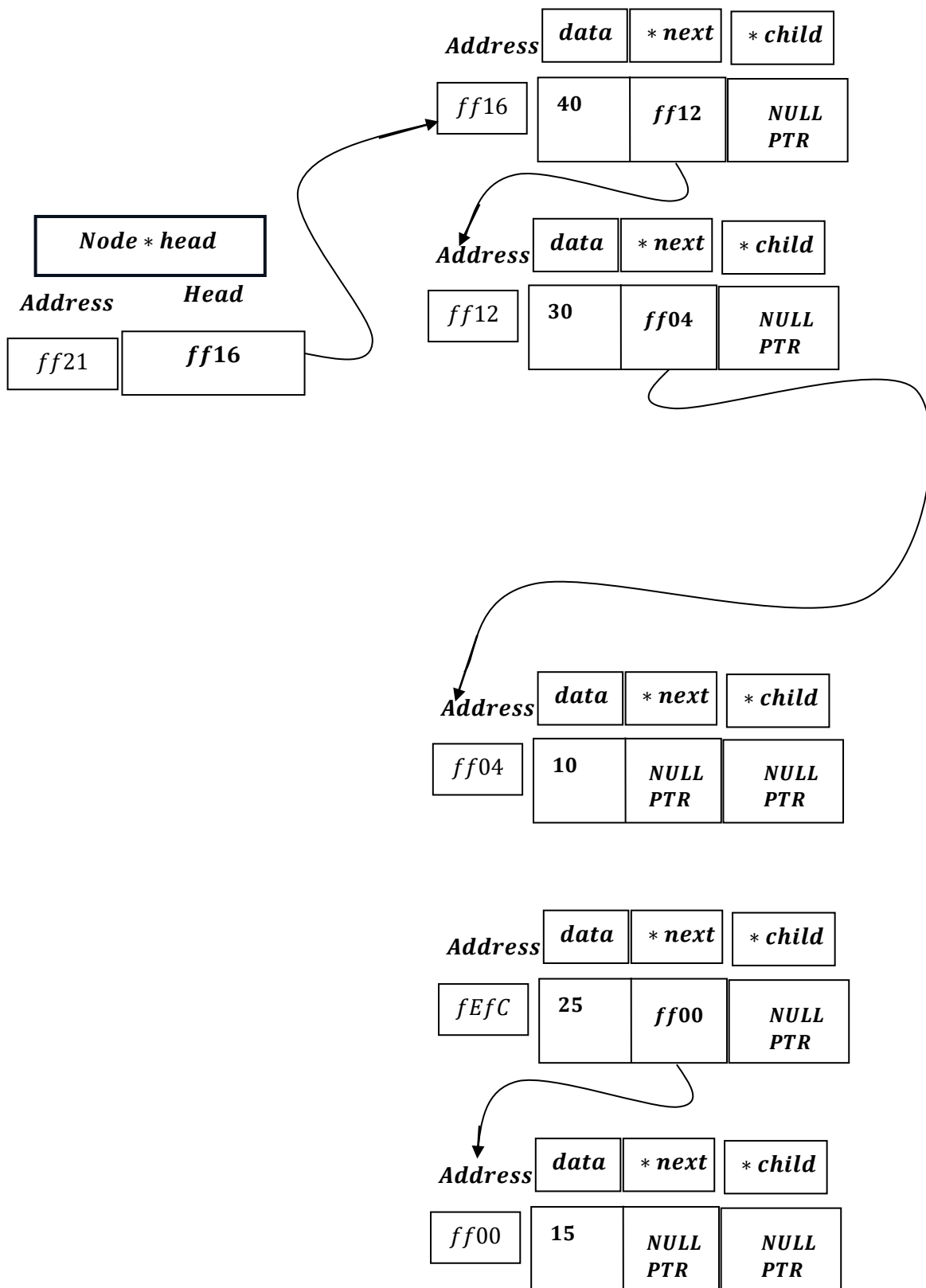
```
cout << "Deleted at position " << pos << endl;
```

Print ``Deleted at position`` : 3

After function gets finished ,



And we are left with:



Note: Though it able to delete node in Main List no doubt , but doesn't able to delete child nodes attached to it.

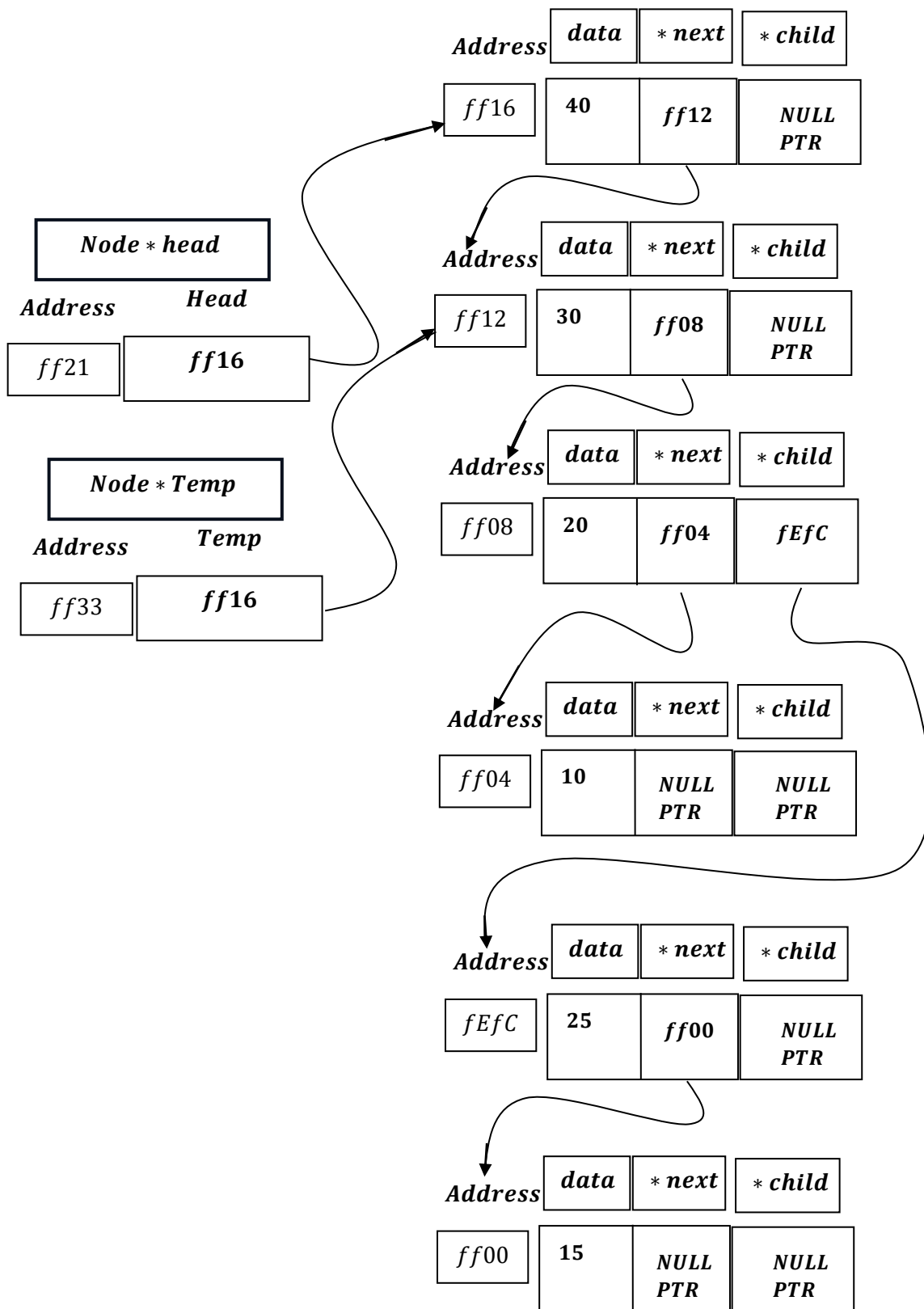
[Now for `` out of Range``]

Say, pos = 5.

Loop traversal:

int i = 1; i < 5 - 1[i < 4 or i = 3] AND TEMP ≠ NULL:

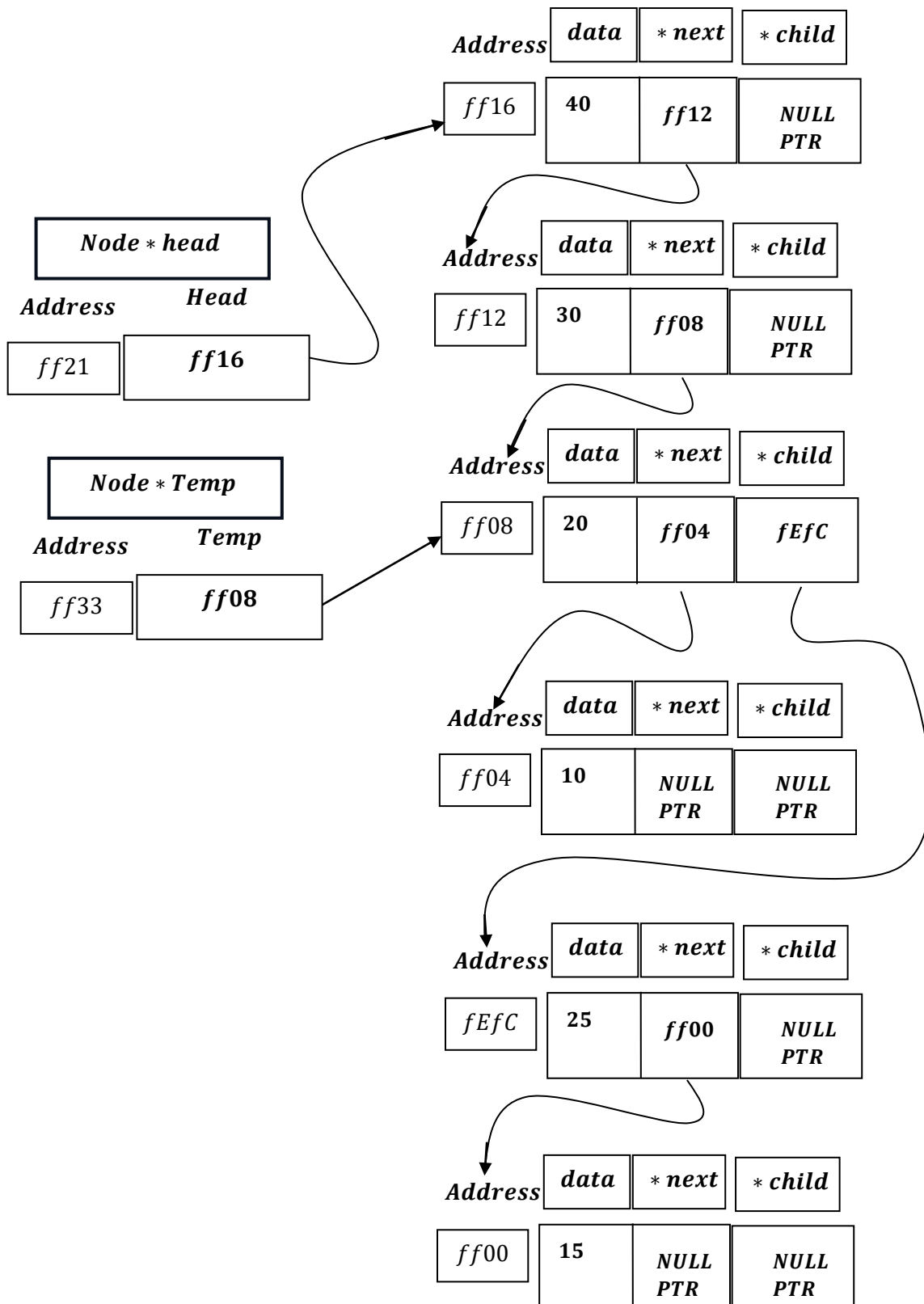
TEMP = TEMP → NEXT = ff12



Update: $i = i + 1 = 2$

$int\ i = 2; i < 5 - 1[i < 4\ or\ i = 3]\ AND\ TEMP \neq NULL:$

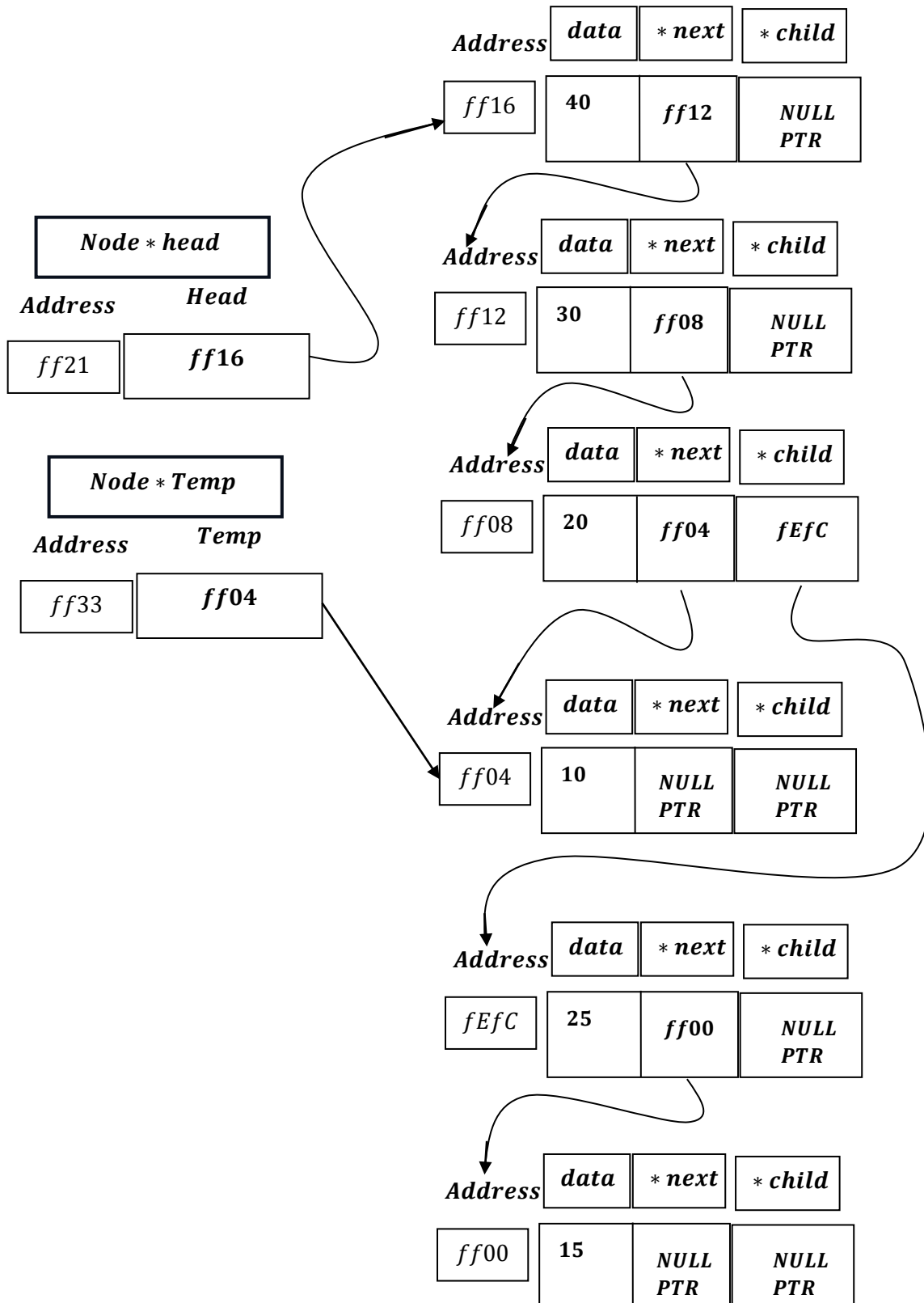
$TEMP = TEMP \rightarrow NEXT = ff08$



Update: $i = i + 1 = 3$

$\text{int } i = 3; i < 5 - 1 [i < 4 \text{ or } i = 3] \text{ AND } \text{TEMP} \neq \text{NULL}:$

$\text{TEMP} = \text{TEMP} \rightarrow \text{NEXT} = \text{ff04}$



Update: $i = i + 1 = 4$

$\text{int } i = 4; i < 5 - 1 [i < 4 \text{ or } i = 3] \text{ AND } TEMP \neq NULL:$

Condition is false , as Lower Bound $>$ Upper Bound, loop exits.

$TEMP \rightarrow NEXT: NULLPTR$, and if we go further:

```
Node *toDelete = temp->next;  
temp->next = toDelete->next;  
free(toDelete);
```

$ToDelete = Temp \rightarrow Next = NULLPTR$

$TEMP \rightarrow Next = ToDelete \rightarrow NEXT$ $\left[\begin{array}{l} \text{Segmentation fault} \\ \text{or Runtime Crash} \\ \text{as, ToDelete is never assigned} \\ \text{and ToDelete} \rightarrow \text{Next,} \\ \text{is never accessed.} \end{array} \right]$

$Free(ToDelete : NULLPTR)$ [Nothing To Free Memory]

Also, Pos: 5 never exists.

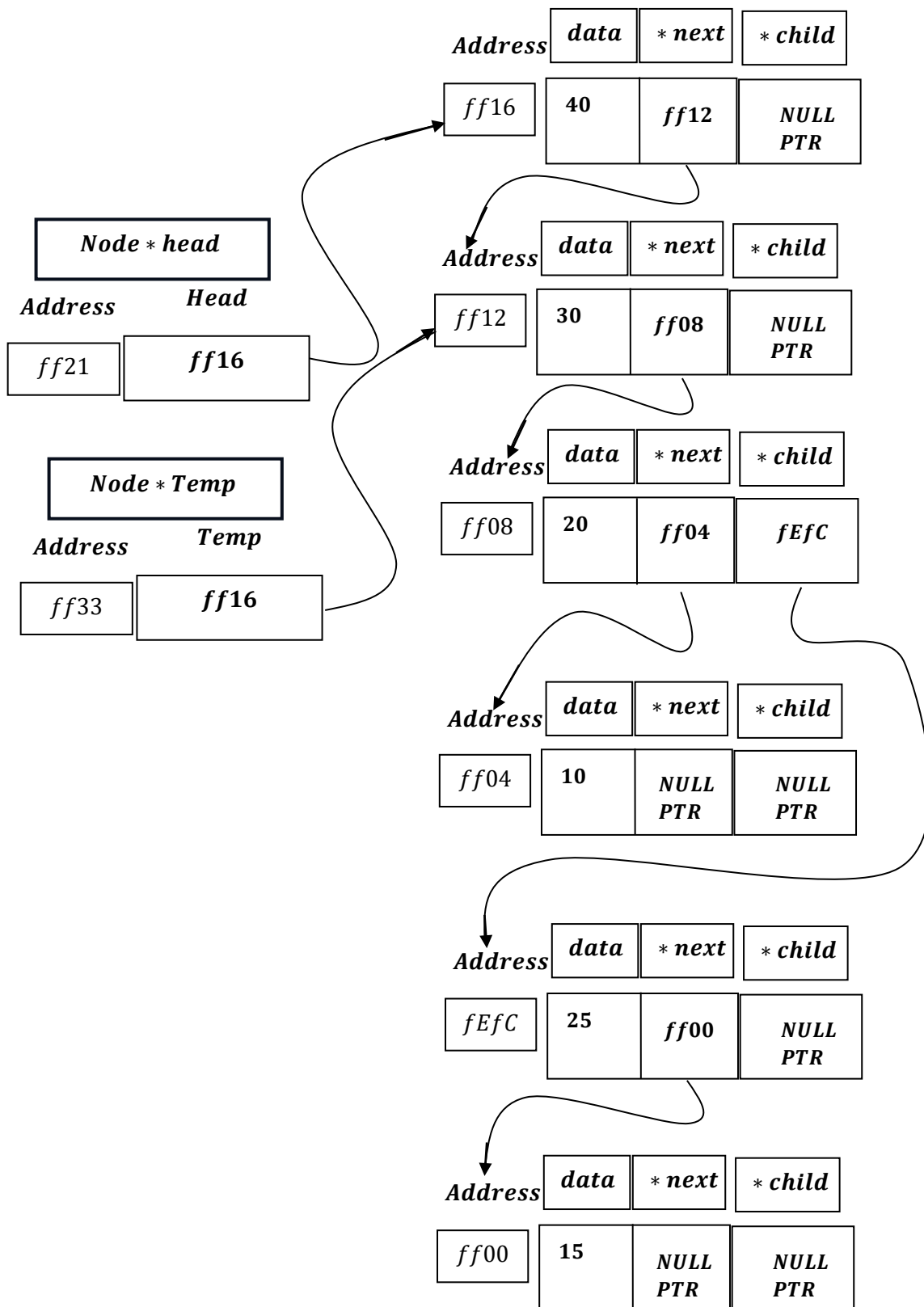
Therefore $Temp \rightarrow Next = NULLPTR$, also ``Out of Range``.

Also if pos = 6, then:

Loop traversal:

$\text{int } i = 1; i < 6 - 1 [i < 5 \text{ or } i = 4] \text{ AND } TEMP \neq NULL:$

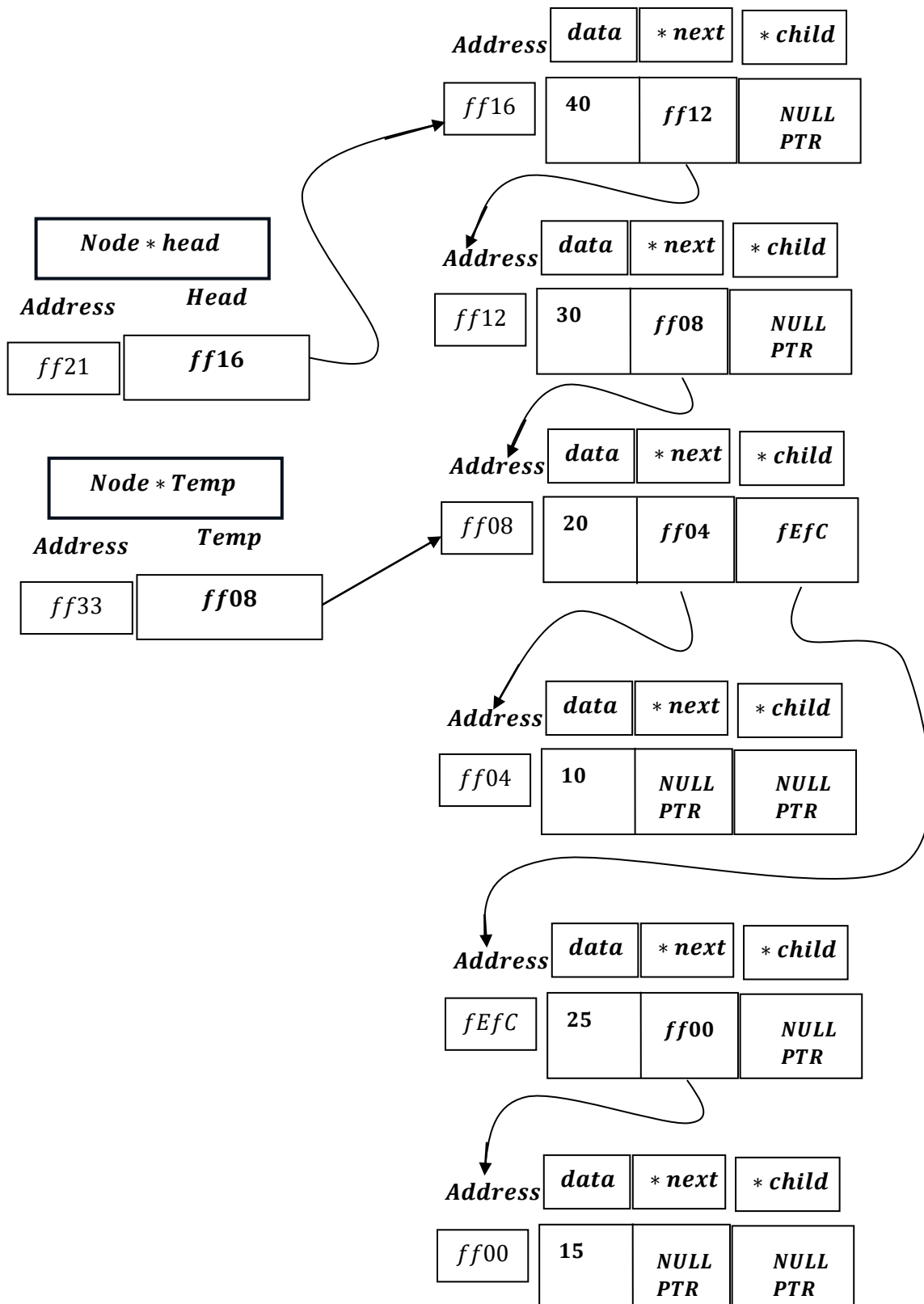
$TEMP = TEMP \rightarrow NEXT = \text{ff12}$



Update: $i = i + 1 = 2$

$int\ i = 2; i < 6 - 1[i < 5\ or\ i = 4]\ AND\ TEMP \neq NULL:$

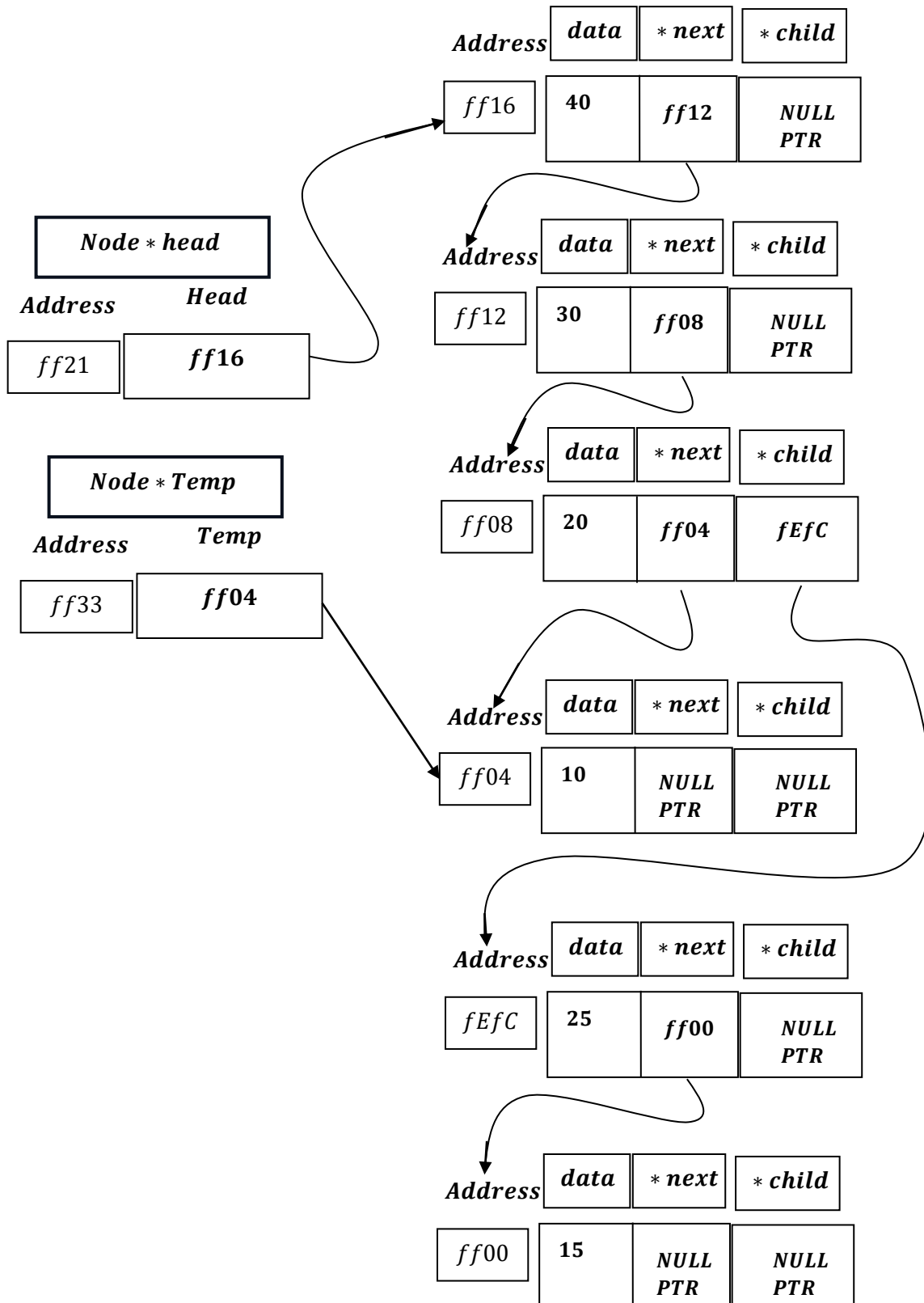
$TEMP = TEMP \rightarrow NEXT = ff08$



Update: $i = i + 1 = 3$

$\text{int } i = 3; i < 6 - 1 [i < 5 \text{ or } i = 4] \text{ AND } \text{TEMP} \neq \text{NULL}:$

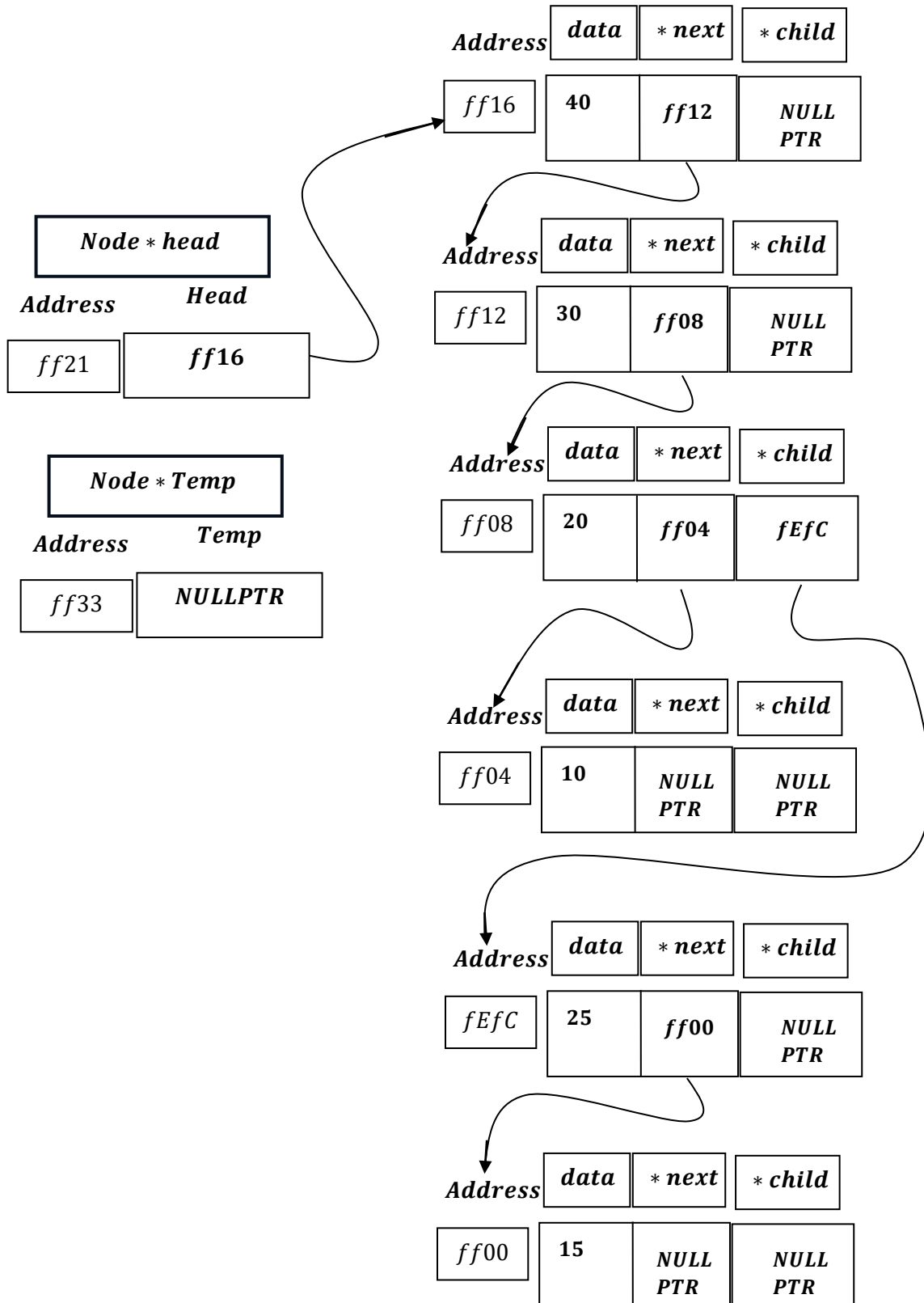
$\text{TEMP} = \text{TEMP} \rightarrow \text{NEXT} = \text{ff04}$



Update: $i = i + 1 = 4$

$\text{int } i = 4; i < 6 - 1 [i < 5 \text{ or } i = 4] \text{ AND } \text{TEMP} \neq \text{NULL}:$

$\text{TEMP} = \text{TEMP} \rightarrow \text{NEXT} = \text{NULLPTR}$



Update: $i = i + 1 = 5$

$\text{int } i = 5; i < 6 - 1 [i < 5 \text{ or } i = 4] \text{ AND } \text{TEMP} : \text{NULL} \neq \text{NULL} [\text{False}]$.

***Condition is false , as Lower Bound > Upper Bound,
and $\text{TEMP} = \text{NULL}$, hence ,loop exits.***

$\therefore$ We can say,

$\text{TEMP} = \text{NULL}$ and $\text{TEMP} \rightarrow \text{NEXT} = \text{NULL}$,position out of range.

```
if (temp == nullptr || temp->next == nullptr)
{
    cout << "Position out of range." << endl;
    return;
}
```

It prints: ``Position out of range.``

Return void and exit from the function.
